Hospital Emergency Room Analytics

Exploratory Data Analysis (EDA) Project

Project Domain: Healthcare Analytics
Project Type: Exploratory Data Analysis
Tools Used: Python, Pandas, NumPy, Matplotlib, Seaborn, Google Colab
Dataset: Hospital Emergency Room Dataset
Dataset Size: 9,216 records and 12 original columns

---

1. Project Overview

Healthcare organizations generate large amounts of data through patient visits, admissions, referrals, and hospital operations. Analyzing this data can help hospitals understand patient flow, identify operational challenges, and make informed decisions.

This project focuses on performing Exploratory Data Analysis (EDA) on hospital emergency room data. The analysis examines patient demographics, admission patterns, department referrals, waiting times, and patient satisfaction.

The project uses Python libraries to clean, explore, summarize, and visualize the dataset. The results are intended to provide insights into emergency room operations and support data-driven decision-making.

Project Motto

"Transforming Hospital Data into Actionable Insights for Better Patient Care and Operational Efficiency."

---

2. Problem Statement

Emergency departments are important components of healthcare organizations. They handle patients with different medical needs, and their operational efficiency can influence the overall patient experience.

Hospital management needs to understand:

- How many patients visit the emergency department?
- Which patient age groups visit most frequently?
- What percentage of patients are admitted?
- Which departments receive the highest number of referrals?
- How long do patients wait before receiving care?

- How does patient satisfaction vary across different groups?
- Are there identifiable patterns in patient visits over time?

This project uses exploratory data analysis to investigate these questions and identify patterns in the available hospital data.

---

3. Project Objectives

The primary objectives of this project are:

1. Understand the structure and characteristics of the hospital dataset.
2. Identify and handle missing values.
3. Check for duplicate records and data quality issues.
4. Analyze patient demographics.
5. Examine patient admission patterns.
6. Explore department referral distributions.
7. Analyze patient waiting times.
8. Investigate patient satisfaction scores.
9. Explore relationships between patient characteristics and hospital operations.
10. Develop meaningful business insights using data visualization.

---

4. Dataset Description

The project uses a publicly available Hospital Emergency Room dataset.

The dataset contains patient emergency room visit records and includes demographic information, admission details, department referrals, waiting times, and satisfaction scores.

Dataset Information

Attribute	Description
Dataset name	Hospital ER Data
Total records	9,216
Original columns	12
File format	CSV
Analysis environment	Google Colab
Data type	Structured tabular data

Dataset Columns

Column	Description
Patient Id	Unique identifier associated with a patient record
Patient Admission Date	Date and time of the patient visit
Patient First Inital	Patient first-name initial
Patient Last Name	Patient last name
Patient Gender	Gender category recorded in the dataset
Patient Age	Age of the patient
Patient Race	Patient race category
Department Referral	Department to which the patient was referred
Patient Admission Flag	Boolean indicator of admission status
Patient Satisfaction Score	Recorded satisfaction score
Patient Waittime	Patient waiting time in minutes
Patients CM	Additional dataset field whose meaning should be confirmed from its source documentation

Note: The original dataset column names are retained to maintain consistency with the source data.

---

5. Tools and Technologies

Python

Python is used as the primary programming language for data analysis.

Google Colab

Google Colab provides a cloud-based Python notebook environment. It allows the project to be completed without installing Python or Jupyter Notebook locally.

Pandas

Pandas is used for:

- Loading CSV files.
- Data manipulation.
- Missing value analysis.
- Grouping and aggregation.

- Statistical summaries.
- Exporting cleaned datasets.

NumPy

NumPy supports numerical calculations and mathematical operations.

Matplotlib

Matplotlib is used to create charts and customize visualizations.

Seaborn

Seaborn is used to create statistical visualizations, including:

- Histograms.
- Bar charts.
- Boxplots.
- Distribution plots.
- Line charts.

---

6. Project Workflow

The project follows a structured data analysis workflow.

1. Dataset collection.
2. Dataset loading.
3. Data understanding.
4. Data quality assessment.
5. Data cleaning and preprocessing.
6. Exploratory Data Analysis.
7. Statistical analysis.
8. Data visualization.
9. Business insights.
10. Exporting the cleaned dataset.

The cleaned dataset can subsequently be used for SQL analysis and dashboard development in Power BI and Tableau.

---

7. Dataset Loading and Initial Inspection

The first step was importing the required Python libraries and loading the CSV file into Google Colab.

Importing Libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
```

Explanation

- `pandas` is imported as `pd`.
- `numpy` is imported as `np`.
- `matplotlib.pyplot` is imported as `plt`.
- `seaborn` is imported as `sns`.

These libraries provide the functionality required for data manipulation, statistical analysis, and visualization.

Loading the Dataset

```
df = pd.read_csv("Hospital ER_Data.csv")
```

The `read_csv()` function reads the CSV file and converts it into a Pandas DataFrame.

A DataFrame is a two-dimensional tabular data structure containing rows and columns.

The variable `df` represents the complete hospital dataset.

Displaying the First Five Records

```
df.head()
```

The `head()` method displays the first five records by default.

This helps inspect the dataset structure and understand the values stored in each column.

Displaying the Last Five Records

```
df.tail()
```

The `tail()` method displays the last five records.

Checking Dataset Dimensions

```
df.shape
```

The `shape` attribute returns a tuple containing the number of rows and columns.

Expected dataset dimensions:

```
(9216, 12)
```

This indicates that the dataset contains 9,216 records and 12 original columns.

Difference Between `shape` and `shape()`

`shape` is an attribute, not a method.

Correct:

```
df.shape
```

Incorrect:

```
df.shape()
```

Calling `shape()` results in a TypeError because the returned tuple is not callable.

Displaying Column Names

```
df.columns
```

This returns the names of all columns in the DataFrame.

Checking Data Types

```
df.info()
```

The `info()` method provides:

- Column names.
- Number of non-null values.
- Data types.
- Memory usage.

This information helps identify potential data quality issues.

Statistical Summary

```
df.describe()
```

The `describe()` method provides statistical summaries for numerical columns.

The output includes:

- Count.
- Mean.
- Standard deviation.
- Minimum.
- Maximum.
- Quartiles.

For categorical columns:

```
df.describe(include="object")
```

This provides information such as the number of unique categories and the most frequently occurring category.

---

8. Data Cleaning and Preprocessing

Data cleaning is an important stage of the data analysis process.

The purpose of data cleaning is to identify missing values, duplicates, incorrect data types, and potential inconsistencies before performing analysis.

8.1 Missing Value Analysis

```
df.isnull().sum()
```

The `isnull()` method identifies missing values.

The `sum()` method counts the missing values in each column.

Missing Value Percentage

```
missing_percentage = (
    df.isnull().sum() / len(df)
) * 100

missing_percentage
```

The formula used is:

Missing Value Percentage = (Missing Values / Total Records) × 100

Missing Value Findings

Column	Missing Values	Missing Percentage
Department Referral	5,400	58.59%
Patient Satisfaction Score	6,699	72.69%
Other columns	0	0%

The Department Referral and Patient Satisfaction Score columns contain missing values.

8.2 Handling Missing Department Referrals

The missing department referral values were replaced with `"No Referral"`.

```
df["Department Referral"] = (
    df["Department Referral"].fillna("No Referral")
)
```

Explanation

The `fillna()` method replaces missing values with a specified value.

In this project, `"No Referral"` is used to represent missing department referral information.

This interpretation should be verified against the dataset's source documentation before treating all missing referrals as confirmed non-referrals.

8.3 Handling Patient Satisfaction Scores

The Patient Satisfaction Score column contains 6,699 missing values.

These missing values were retained rather than being replaced with an arbitrary average.

Reason

Replacing missing satisfaction scores with an average could distort the actual distribution of patient satisfaction.

The analysis therefore uses only the available satisfaction scores.

8.4 Duplicate Record Analysis

```
df.duplicated().sum()
```

The `duplicated()` method identifies duplicate rows.

The `sum()` method counts the number of duplicate records.

Finding

The dataset contains zero duplicate records.

Therefore, no duplicate rows were removed.

8.5 Date Conversion

The Patient Admission Date column was converted into a datetime data type.

```
df["Patient Admission Date"] = pd.to_datetime(
    df["Patient Admission Date"],
    format="%d-%m-%Y %H:%M",
    errors="coerce"
)
```

Explanation

`pd.to_datetime()` converts date strings into datetime objects.

The format:

- `%d` represents the day.
- `%m` represents the month.
- `%Y` represents the four-digit year.
- `%H` represents the hour.
- `%M` represents the minute.

The `errors="coerce"` parameter converts invalid date values into `NaT` (Not a Time).

This enables time-based analysis, including monthly visit patterns.

8.6 Verifying Missing Values

```
df.isnull().sum()
```

After cleaning the Department Referral column, the remaining missing values are in Patient Satisfaction Score.

These missing values are retained for analysis.

---

9. Understanding Categorical Variables

Categorical variables represent groups or classifications.

The unique values in important categorical columns were examined.

Gender Categories

```
df["Patient Gender"].unique()
```

The dataset contains:

```
M
F
NC
```

These categories are retained as recorded in the source dataset.

Admission Status

```
df["Patient Admission Flag"].unique()
```

The dataset contains:

```
False
True
```

The Boolean values represent admission status.

- `True` indicates an admitted patient.
- `False` indicates a patient who was not admitted.

Department Referral Categories

```
df["Department Referral"].unique()
```

The dataset includes department referral categories such as:

- No Referral.
- General Practice.
- Orthopedics.
- Gastroenterology.
- Physiotherapy.
- Neurology.
- Cardiology.
- Renal.

Patient Age Range

```
print("Minimum Age:", df["Patient Age"].min())
print("Maximum Age:", df["Patient Age"].max())
```

The dataset contains patient ages ranging from 1 to 79 years.

---

10. Exploratory Data Analysis

Exploratory Data Analysis is the process of examining and understanding data using statistical methods and visualizations.

EDA helps identify patterns, trends, distributions, and relationships between variables.

The following analyses were performed.

10.1 Patient Gender Distribution

Business Question

How are patients distributed across gender categories?

Code

```
gender_counts = df["Patient Gender"].value_counts()

plt.figure(figsize=(7, 5))

sns.barplot(
    x=gender_counts.index,
    y=gender_counts.values
)

plt.title("Patient Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Number of Patients")

plt.show()
```

Explanation

`value_counts()` calculates the number of records belonging to each gender category.

The bar chart visualizes the distribution of patient visits across gender categories.

Business Purpose

This analysis helps understand the demographic composition of emergency room visits.

10.2 Patient Age Distribution

Business Question

Which age groups account for the largest number of emergency room visits?

Code

```
plt.figure(figsize=(10, 5))

sns.histplot(
    data=df,
    x="Patient Age",
    bins=20,
    kde=True
)

plt.title("Patient Age Distribution")
plt.xlabel("Patient Age")
plt.ylabel("Number of Patients")

plt.show()
```

Explanation

`data=df` specifies the DataFrame containing the data.

`x="Patient Age"` specifies the column to be plotted on the X-axis.

`bins=20` divides the age range into 20 intervals.

`kde=True` adds a smooth density curve to help understand the distribution.

Business Purpose

This analysis helps identify the age groups that account for the largest number of emergency room visits.

10.3 Patient Admission Status

Business Question

What proportion of emergency room visits result in hospital admission?

Code

```
admission_counts = (
```

```
    df["Patient Admission Flag"].value_counts()
)

plt.figure(figsize=(7, 5))

sns.barplot(
    x=admission_counts.index.astype(str),
    y=admission_counts.values
)

plt.title("Patient Admission Status")
plt.xlabel("Admitted (True/False)")
plt.ylabel("Number of Patients")

plt.show()
```

Explanation

`df["Patient Admission Flag"]` selects the admission status column.

`value_counts()` counts the number of `True` and `False` values.

The bar chart compares admitted and non-admitted patient visits.

10.4 Department Referral Analysis

Business Question

Which departments receive the highest number of patient referrals?

Code

```
department_counts = (
    df["Department Referral"].value_counts()
)

department_counts
```

Visualization

```
plt.figure(figsize=(12, 6))

sns.barplot(
    x=department_counts.index,
    y=department_counts.values
)

plt.title("Patient Referrals by Department")
plt.xlabel("Department")
plt.ylabel("Number of Patients")

plt.xticks(rotation=45)
```

```
plt.show()
```

Explanation

`value_counts()` calculates the number of patient records associated with each department.

The bar chart compares the number of referrals across departments.

Business Purpose

This analysis helps understand how emergency room referrals are distributed across hospital departments.

It may also help identify departments receiving a comparatively large volume of referrals.

10.5 Department Referral Percentage

Code

```
department_percentage = (
    df["Department Referral"].value_counts(
        normalize=True
    ) * 100
)

department_percentage.round(2)
```

Explanation

`normalize=True` returns proportions instead of raw counts.

Multiplying by 100 converts proportions into percentages.

The `round(2)` method rounds the results to two decimal places.

Business Purpose

Percentages make it easier to compare department referral distributions.

10.6 Patient Waiting Time Analysis

Business Question

How long do patients wait before receiving emergency care?

Statistical Analysis

```
print(
    "Average Waiting Time:",
    df["Patient Waittime"].mean()
)

print(
    "Median Waiting Time:",
    df["Patient Waittime"].median()
)

print(
    "Maximum Waiting Time:",
    df["Patient Waittime"].max()
)

print(
    "Minimum Waiting Time:",
    df["Patient Waittime"].min()
)
```

Explanation

- `mean()` calculates the average waiting time.
- `median()` calculates the middle waiting time.
- `max()` identifies the maximum waiting time.
- `min()` identifies the minimum waiting time.

Waiting Time Distribution

```
plt.figure(figsize=(10, 5))

sns.histplot(
    data=df,
    x="Patient Waittime",
    bins=20,
    kde=True
)

plt.title("Patient Waiting Time Distribution")
plt.xlabel("Waiting Time (Minutes)")
plt.ylabel("Number of Patients")

plt.show()
```

The histogram displays the distribution of patient waiting times.

Business Purpose

The analysis helps identify the typical waiting time and the distribution of waiting times across emergency room visits.

10.7 Waiting Time by Admission Status

Business Question

Do admitted patients experience different waiting times compared with non-admitted patients?

Code

```
plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="Patient Admission Flag",
    y="Patient Waittime"
)

plt.title("Waiting Time by Admission Status")
plt.xlabel("Admitted (True/False)")
plt.ylabel("Waiting Time (Minutes)")

plt.show()
```

Explanation

A boxplot displays:

- Median.
- Lower quartile.
- Upper quartile.
- Distribution of values.
- Potential outliers.

Business Purpose

This analysis compares waiting time distributions across admission groups.

The result can help identify differences in waiting time patterns.

10.8 Patient Admission Percentage

Code

```
admission_counts = (
    df["Patient Admission Flag"].value_counts()
)

admission_percentage = (
    df["Patient Admission Flag"].value_counts(
        normalize=True
```

```
    ) * 100
)

print("Admission Counts:")
print(admission_counts)

print("Admission Percentage:")
print(admission_percentage.round(2))
```

Explanation

The admission percentage represents the proportion of emergency room visits resulting in admission.

The Boolean admission flag can be averaged to calculate the proportion of `True` values.

```
admission_rate = (
    df["Patient Admission Flag"].mean() * 100
)
```

Business Purpose

This metric helps understand the proportion of emergency room visits resulting in hospital admission.

10.9 Patient Satisfaction Analysis

Business Question

What is the average patient satisfaction score among records with available satisfaction data?

Code

```
print(
    "Available Satisfaction Scores:",
    df["Patient Satisfaction Score"].count()
)

print(
    "Average Satisfaction Score:",
    df["Patient Satisfaction Score"].mean()
)

print(
    "Median Satisfaction Score:",
    df["Patient Satisfaction Score"].median()
)
```

Explanation

`count()` counts non-missing satisfaction scores.

`mean()` calculates the average satisfaction score.

`median()` calculates the median satisfaction score.

Missing satisfaction values are excluded from these calculations.

Business Purpose

This analysis helps summarize recorded patient satisfaction scores.

Because a large proportion of satisfaction values are missing, the findings should be interpreted cautiously.

10.10 Satisfaction by Admission Status

Code

```
satisfaction_by_admission = (
    df.groupby("Patient Admission Flag")[
        "Patient Satisfaction Score"
    ].mean()
)

satisfaction_by_admission
```

Explanation

`groupby()` groups the records according to admission status.

The `mean()` method calculates the average satisfaction score within each group.

Business Purpose

This analysis explores whether recorded satisfaction scores differ between admitted and non-admitted patients.

The result describes an association and does not establish causation.

---

11. Advanced EDA

Advanced EDA explores relationships between variables and investigates more detailed business questions.

11.1 Waiting Time by Department

Business Question

Which departments have the highest average patient waiting times?

Code

```
department_waittime = (
    df.groupby("Department Referral")[
        "Patient Waittime"
    ].mean()
    .sort_values(ascending=False)
)

department_waittime
```

Visualization

```
plt.figure(figsize=(12, 6))

sns.barplot(
    x=department_waittime.values,
    y=department_waittime.index
)

plt.title("Average Waiting Time by Department")
plt.xlabel("Average Waiting Time (Minutes)")
plt.ylabel("Department")

plt.show()
```

Explanation

`groupby()` groups patient records by department referral.

`mean()` calculates the average waiting time for each group.

`sort_values(ascending=False)` sorts the departments from highest to lowest average waiting time.

Business Purpose

This analysis identifies differences in average waiting time across referral categories.

11.2 Admission Rate by Age Group

Business Question

Which age groups have higher admission rates?

Creating Age Groups

```
df["Age Group"] = pd.cut(
    df["Patient Age"],
    bins=[0, 18, 35, 50, 65, 100],
    labels=[
        "0-18",
        "19-35",
        "36-50",
        "51-65",
        "65+"
    ]
)
```

Explanation

`pd.cut()` divides continuous numerical data into predefined intervals.

The age groups are created to simplify age-based analysis.

Calculating Admission Rate

```
age_admission = (
    df.groupby(
        "Age Group",
        observed=True
    )["Patient Admission Flag"].mean() * 100
)

age_admission
```

Visualization

```
plt.figure(figsize=(9, 5))

sns.barplot(
    x=age_admission.index,
    y=age_admission.values
)

plt.title("Admission Rate by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Admission Rate (%)")

plt.show()
```

Explanation

The admission flag is Boolean.

In Python, `True` is treated as 1 and `False` as 0 during numerical calculations.

Therefore, the average of the admission flag represents the proportion of admitted visits.

Multiplying the average by 100 converts it into a percentage.

Business Purpose

This analysis compares admission rates across different patient age groups.

11.3 Monthly Emergency Room Visits

Business Question

How does emergency room visit volume vary across months?

Extracting Month

```
df["Admission Month"] = (
    df["Patient Admission Date"].dt.month
)
```

The `.dt.month` accessor extracts the numerical month from the datetime column.

For example:

- January = 1
- February = 2
- March = 3

Calculating Monthly Visits

```
monthly_visits = (
    df["Admission Month"]
    .value_counts()
    .sort_index()
)

monthly_visits
```

Explanation

`value_counts()` counts visits for each month.

`sort_index()` arranges the month numbers in ascending order.

This ensures chronological order rather than sorting months by visit frequency.

Visualization

```
plt.figure(figsize=(12, 5))

sns.lineplot(
    x=monthly_visits.index,
    y=monthly_visits.values,
    marker="o"
)

plt.title("Monthly Emergency Room Visits")
plt.xlabel("Month")
plt.ylabel("Number of Visits")

plt.xticks(
    range(1, 13),
    [
        "Jan", "Feb", "Mar", "Apr",
        "May", "Jun", "Jul", "Aug",
        "Sep", "Oct", "Nov", "Dec"
    ]
)

plt.show()
```

Business Purpose

This analysis helps identify differences in emergency room visit volume across months.

Important Note

The monthly visit chart should be interpreted based on the actual date range of the dataset.

A single year's data can describe monthly variation, but it cannot establish a recurring seasonal pattern.

---

12. Key Performance Indicators (KPIs)

KPIs are numerical metrics used to summarize performance.

For this project, the following KPIs were identified.

KPI	Description
Total Patient Visits	Total number of records in the dataset
Admission Rate	Percentage of visits resulting in admission
Average Waiting Time	Average patient waiting time in minutes
Average Patient Age	Average age of patients
Average Satisfaction Score	Average of available satisfaction scores
Most Common Department Referral	Department category with the highest recorded count

KPI Calculation

```
total_visits = len(df)

admission_rate = (
    df["Patient Admission Flag"].mean() * 100
)

average_waittime = (
    df["Patient Waittime"].mean()
)

average_age = (
    df["Patient Age"].mean()
)

average_satisfaction = (
    df["Patient Satisfaction Score"].mean()
)

most_referred_department = (
    df["Department Referral"].value_counts().idxmax()
)
```

KPI Summary

```
insights = pd.DataFrame({
    "KPI": [
        "Total Patient Visits",
        "Admission Rate (%)",
        "Average Waiting Time (Minutes)",
        "Average Patient Age",
        "Average Satisfaction Score",
        "Most Common Department Referral"
    ],
    "Value": [
        total_visits,
```

```
        round(admission_rate, 2),
        round(average_waittime, 2),
        round(average_age, 2),
        round(average_satisfaction, 2),
        most_referred_department
    ]
})

insights
```

This table consolidates the main numerical metrics into one DataFrame.

---

13. Business Questions Addressed

The project investigates the following questions:

No.	Business Question	Analysis Method
1	How many patient visits are recorded?	Dataset summary
2	How are patients distributed by gender?	Value counts and bar chart
3	Which age groups visit most frequently?	Histogram
4	What percentage of patients are admitted?	Admission rate
5	Which departments receive the most referrals?	Grouping and bar chart
6	What is the average waiting time?	Statistical analysis
7	How are waiting times distributed?	Histogram
8	Do waiting times differ by admission status?	Boxplot
9	What is the average recorded satisfaction score?	Statistical analysis
10	Does satisfaction differ by admission status?	Grouped average
11	Which departments have higher average waiting times?	Grouping and bar chart
12	How do admission rates vary by age group?	Age grouping
13	How does visit volume vary across months?	Monthly aggregation and line chart

---

14. Data Export

After completing data cleaning and analysis, the DataFrame was exported as a cleaned CSV file.

```
df.to_csv(
    "Hospital_ER_Cleaned.csv",
    index=False
)
```

Explanation of `index=False`

Pandas assigns an index to every row in a DataFrame.

For example, the index may contain:

```
0
1
2
3
4
```

The parameter `index=False` prevents this additional DataFrame index from being written into the CSV file.

It does not delete any original rows or columns.

Downloading the Cleaned Dataset

```
from google.colab import files

files.download("Hospital_ER_Cleaned.csv")
```

This downloads the cleaned dataset from Google Colab to the local computer.

The cleaned CSV can be used for SQL analysis and dashboard development.

---

15. Project Limitations

The following limitations should be considered when interpreting the analysis.

Missing Satisfaction Data

A large proportion of patient satisfaction scores are missing.

Therefore, satisfaction analysis is based only on available records and may not represent every patient visit.

Department Referral Interpretation

Missing department referral values were replaced with `"No Referral"`.

This interpretation should be validated against the source dataset documentation.

Limited Dataset Scope

The analysis is based on the available dataset and does not represent all hospitals or emergency departments.

The findings should not be generalized to every healthcare organization.

Descriptive Analysis

The project focuses on descriptive and exploratory analysis.

The findings identify patterns and associations but do not establish causation.

No Medical Diagnosis

The project analyzes operational and demographic patterns.

It does not provide medical diagnoses, treatment recommendations, or clinical decision-making.

Privacy Considerations

The original dataset includes patient identifiers and name-related fields.

These fields should not be displayed in public dashboards or GitHub documentation.

The project should use aggregated data and avoid exposing identifiable patient information.

---

16. Future Scope

The project can be extended in several ways.

MySQL Analysis

The cleaned dataset can be imported into MySQL.

SQL queries can be used to answer business questions involving:

- Patient counts.
- Admission percentages.
- Department referrals.
- Average waiting times.
- Age-group analysis.
- Monthly patient visits.

Power BI Dashboard

An interactive Power BI dashboard can be created with:

- Total patient visits.
- Admission rate.
- Average waiting time.
- Patient demographics.
- Department referral distribution.
- Monthly visit trends.
- Patient satisfaction metrics.

Tableau Dashboard

A Tableau dashboard can be developed to explore the same dataset using interactive visualizations.

The Power BI and Tableau dashboards can demonstrate different approaches to data visualization and storytelling.

Additional Analysis

Further analysis could investigate:

- Patient arrival patterns by day of the week.
- Admission patterns by age and gender.
- Waiting time distribution by department.
- Satisfaction score distribution.
- Potential operational bottlenecks.

Any additional analysis should be based on the available data and should avoid unsupported conclusions.

---

17. Conclusion

This project demonstrates the application of Python-based exploratory data analysis to hospital emergency room data.

The project includes dataset loading, data understanding, missing value analysis, duplicate checking, data type conversion, categorical analysis, statistical summaries, and data visualization.

The analysis explores patient demographics, admission patterns, department referrals, waiting times, patient satisfaction, and monthly visit distributions.

The use of Pandas, NumPy, Matplotlib, and Seaborn provides a structured approach to understanding the dataset and identifying patterns.

The project also establishes a foundation for further analysis using MySQL, Power BI, and Tableau.

By transforming raw hospital data into meaningful information, this project demonstrates how data analytics can support operational understanding and data-driven decision-making in healthcare.

---

18. Repository Structure

The following structure can be used when uploading the project to GitHub.

```
Hospital-Healthcare-Analytics-EDA/
│
├── Hospital_Healthcare_Analytics_EDA_Project.ipynb
│
├── Hospital_ER_Cleaned.csv
│
├── README.md
│
├── Hospital_EDA_Documentation.md
│
├── visualizations/
│   ├── patient_gender_distribution.png
│   ├── patient_age_distribution.png
│   ├── patient_admission_status.png
│   ├── department_referral_analysis.png
│   ├── waiting_time_distribution.png
│   └── monthly_emergency_room_visits.png
└── sql/
    └── hospital_analysis.sql
```

Note: The CSV file should only be uploaded if its license permits redistribution and it does not contain sensitive or identifiable patient information. Otherwise, provide instructions for obtaining the original dataset and use an appropriately cleaned, non-identifiable version.

---

19. Project Summary

Project: Hospital Emergency Room Analytics

Domain: Healthcare Analytics

Primary Objective: Explore hospital emergency room data to understand patient demographics, admissions, department referrals, waiting times, and satisfaction.

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn, Google Colab.

Methods: Data cleaning, descriptive statistics, exploratory data analysis, grouping, aggregation, and data visualization.

Expected Outcome: A portfolio-ready healthcare analytics project demonstrating data analysis skills and providing a foundation for SQL, Power BI, and Tableau.
